

Frontend – Assignment 1

Madhur Bhardwaj | B.Tech (Cloud Computing), 2nd Year | GLA University

Q1. HTTP Request–Response Cycle

Every time a browser shows you a webpage, it's really just running a conversation with a server, made up of a request and a response. Here's what happens step by step:

- **DNS Lookup:** The browser first needs to know the actual address of the server, so it translates the domain name (like example.com) into an IP address using DNS.
- **Establishing a Connection:** Once it has the IP, the browser opens a TCP connection to the server, usually over HTTPS, which also involves a TLS handshake to secure the channel.
- **Sending the Request:** The browser sends an HTTP request to the server. This includes a method (GET, POST, etc.), the path of the resource being asked for, and headers carrying extra information like cookies or the type of content the browser can accept.
- **Server Processing:** The server receives the request, figures out what's being asked for, may talk to a database or run some backend logic, and prepares a response.
- **Sending the Response:** The server sends back an HTTP response, which contains a status code (200 OK, 404 Not Found, 500 Server Error, etc.), response headers, and the actual content (HTML, JSON, images, etc.).
- **Rendering:** The browser receives the response and renders it, parsing the HTML, applying CSS, and running any JavaScript to build the final page the user sees.

A simple way to picture this as a diagram:

Browser (Client) → DNS Lookup → TCP/TLS Connection → HTTP Request → Web Server → HTTP Response → Browser renders page

This request–response cycle repeats for every resource on a page — the initial HTML document, then separate requests for CSS files, JavaScript files, images, and any data fetched dynamically.

Q2. Introduction to Web Technologies

1. HTML

HTML (HyperText Markup Language) is the backbone of every webpage — it defines the structure and content, like headings, paragraphs, links, images, and forms. It isn't a programming language; it's a markup language that tells the browser what each piece of content is.

2. CSS

CSS (Cascading Style Sheets) handles how a page looks. It controls colors, fonts, spacing, layout, and responsiveness, letting you separate the visual design from the HTML structure so the same content can look completely different with a new stylesheet.

3. JavaScript

JavaScript is what makes a page interactive. It runs in the browser and can respond to clicks, validate forms, update content without reloading the page, fetch data from a server, and much more. It's the only language that runs natively in every browser.

4. Bootstrap

Bootstrap is a front-end framework built on top of HTML, CSS, and JavaScript. It gives developers a ready-made grid system, pre-styled components (buttons, navbars, modals, cards), and responsive utilities, so you don't have to build common UI patterns from scratch.

5. jQuery

jQuery is a JavaScript library that simplifies common tasks like DOM manipulation, event handling, animations, and AJAX calls. It became popular because it smoothed over browser inconsistencies with a short, easy-to-read syntax at a time when writing plain JavaScript across browsers was painful.

6. React.js

React.js is a JavaScript library for building user interfaces, developed by Facebook. It lets developers build UIs out of reusable components and uses a virtual DOM to update only the parts of the page that actually change, which makes complex, data-driven interfaces faster and easier to manage.

Q3. Client-Side vs. Server-Side Technologies

Client-side technologies are the ones that run directly in the user's browser. HTML, CSS, and JavaScript are the classic examples — they control what the user sees and how the page behaves on their own device, without needing to contact the server again for every little interaction.

Server-side technologies, on the other hand, run on the web server, away from the user's machine. They handle things like talking to databases, authenticating users, and generating dynamic content before sending the final result to the browser. Examples include Node.js, PHP, Python (Django/Flask), Java (Spring), and ASP.NET.

The key difference is where the code executes and what it has access to. Client-side code is visible to the user and can be viewed through 'Inspect Element,' while server-side code stays hidden on the server and is what usually handles sensitive logic like passwords or payments. In a typical web app, both work together — the server sends data, and the client-side code decides how to display and interact with it.

Q4. Block-Level Elements in HTML

Block-level elements are HTML elements that always start on a new line and take up the full width available to them, pushing any content after them down to the next line. They're used to structure the page into distinct sections.

Key characteristics:

- They start on a new line by default.
- They stretch to take up the full width of their parent container, regardless of the content inside.
- They can contain other block-level elements as well as inline elements.
- You can set width, height, margin, and padding on them freely.

Examples of block-level elements:

- `<div>` — a generic container used for grouping content.
- `<p>` — used for paragraphs of text.
- `<h1>` to `<h6>` — heading elements.
- Other common examples include `<ul>`, `<table>`, and `<form>`.

Q5. Semantic HTML Elements

Semantic elements are HTML tags that clearly describe their meaning and purpose, both to the browser and to anyone reading the code — as opposed to generic tags like `<div>` and `<span>`, which say nothing about the content inside them.

Why they matter:

- **Accessibility:** Screen readers rely on semantic tags to help visually impaired users navigate a page more easily, understanding what's a navigation menu versus what's the main content.
- **SEO:** Search engines use semantic structure to understand a page's content better, which can improve how it ranks and is indexed.
- **Readability:** Code that uses `<header>` or `<footer>` instead of endless generic `<div>` tags is much easier for other developers to read and maintain.

Common examples: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, and `<footer>`. For instance, `<nav>` should wrap the site's navigation links, and `<article>` should wrap a self-contained piece of content like a blog post.

Q6. Creating a Table Using HTML

Here's the HTML code to create the employee table:

```
<table border="1">
  <tr>
    <th>Sr. No.</th>
    <th>Name</th>
    <th>Designation</th>
    <th>Department</th>
  </tr>
  <tr>
    <td>1</td>
    <td>John</td>
```

```
<td>Manager</td>
<td>Sales</td>
</tr>
<tr>
<td>2</td>
<td>Smith</td>
<td>Sr. Manager</td>
<td>Marketing</td>
</tr>
<tr>
<td>3</td>
<td>Jane</td>
<td>Manager</td>
<td>HR</td>
</tr>
</table>
```

Q7. Creating a Nested List Using HTML

Since 'River' is one heading with several tributaries listed under it, an unordered list nested inside another unordered list is the right choice here:

```
<ul>
<li>River
<ul>
<li>Ganga</li>
<li>Yamuna</li>
<li>Brahmaputra</li>
<li>Narmada</li>
</ul>
</li>
</ul>
```

Q8. Types of CSS

There are three main ways to apply CSS to an HTML document:

Inline CSS

Styles are written directly inside an HTML tag using the style attribute. It's quick, but it mixes content and design, so it's hard to maintain for anything beyond a one-off tweak.

Example: `<p style="color: blue;">Hello</p>`

Internal (Embedded) CSS

Styles are written inside a `<style>` tag within the `<head>` section of the HTML document. This keeps the CSS in one place, which is useful for single-page projects, but it doesn't carry over to other pages.

Example: `<style> p { color: blue; } </style>`

External CSS

Styles are written in a separate .css file and linked to the HTML document using a `<link>` tag. This is the most practical approach for real projects since one stylesheet can style multiple pages, and it keeps content and design cleanly separated.

Example: `<link rel="stylesheet" href="style.css">`

Q9. Practical Understanding

HTML, CSS, and JavaScript each play a distinct role in building a webpage, and a simple way to remember the split is: HTML is the structure, CSS is the style, and JavaScript is the behavior.

- HTML builds the skeleton of the page — it defines what content exists, like headings, paragraphs, images, and buttons, without worrying about how any of it looks.
- CSS takes that skeleton and dresses it up, controlling colors, fonts, spacing, alignment, and how the layout adapts across different screen sizes.
- JavaScript brings the page to life, letting it respond to user actions, update content dynamically, validate inputs, and communicate with servers without needing a full page reload.

Put together, a static HTML page becomes a styled, interactive website only once CSS and JavaScript are layered on top of it.

Q10. Short Answer

- a. Role of Bootstrap: Bootstrap speeds up front-end development by providing a pre-built, responsive grid system and a library of ready-made UI components like navbars, buttons, forms, and modals. Instead of writing custom CSS for common patterns, developers can just apply Bootstrap's classes and get a consistent, mobile-friendly design out of the box.
- b. What jQuery is and why it was widely used: jQuery is a lightweight JavaScript library that simplifies DOM manipulation, event handling, animations, and AJAX requests. It became hugely popular in the mid-2000s to 2010s because it offered a short, consistent syntax that worked reliably across different browsers, at a time when plain JavaScript behaved inconsistently between them.
- c. What React.js is and the problem it solves: React.js is a JavaScript library for building user interfaces out of reusable, self-contained components. It solves the problem of efficiently updating complex, data-driven UIs by using a virtual DOM — instead of reloading or manually updating the whole page, React figures out exactly what changed and updates only that part, making interactive applications faster and easier to build and maintain.